Peer review: Egyptian Civil Code RAG

Repository: https://github.com/AhmedYasser06/Egyptian-Civil-Code-RAG
Date: 16 October 2026

Hi! Thank you for sharing this project. The folder structure (`retrieval`, `llm`, `schemas`, `routes`, `serving`, `monitoring`, `evaluation`) made it easy to find my way around, and it is clear that a lot of thought went into making this more than a notebook demo. Below is my review, written as honestly and kindly as I can. I only wrote about things I saw myself while running or reading the project.

1. Setup: did it run, how long, what broke?

My environment. A Windows machine with about 4 GB of RAM, Ubuntu on WSL (WSL could use about 1.8 GB of that), Docker Desktop and a Python virtual environment. I cloned the repository and followed the README quick-start steps in order.

What happened, step by step:

- `cp .env.example .env` and adding a Groq API key: no problems.
- `pip install numpy pyyaml && python -m src.scripts.fetch_data` (inside a virtual environment) worked straight away. It printed `articles=1149 chunks=2298 embedded=2298` and `final-articles md5 matches dvc.lock: True`. That check was reassuring, and not needing a DVC remote is a nice touch.
- My first `docker compose up --build` stopped immediately with "Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?" Docker was simply not running on my machine. After starting it, I ran the command again.
- The build took about 40 minutes in total.
- Right after the build, the API container exited with code 1 and restarted. The log showed a `ReadTimeout` from the Qdrant client inside `src/scripts/ensure_index.py` (line 54, `ensure_collection`) while creating the collection. The log then finished with `[ensure_index] indexed 2298 chunks into 'egyptian_civil_code'`.
- Once both containers were up, `docker compose ps` showed the API as `(healthy)`, and `/health` returned `{"status":"healthy","documents_indexed":1149}`. The interactive docs page at `/docs` loaded, and the OpenAPI schema included `/ask`, `/ask/stream`, `/drift` and `/health`. Opening `/` in the browser returned a 404, which is normal for an API, but a redirect to `/docs` would be friendlier.
- I sent one question to `POST /ask`. It had not returned after 3 min 21 s, when I stopped it. Because of that, this review is about setup and code rather than answer quality.

Friction points, in short: Docker needs to be running before the first command; the first build is long (about 40 minutes for me); the API crashed once right after the build because of a Qdrant timeout, then started again; and `/` returns a 404.

What was clear: The quick-start is short and the commands are in a sensible order. One `docker compose up` brings up both Qdrant and the API. The data step prints counts and a checksum, so you know straight away that it worked.

2. Code quality: two patterns

Good pattern: a pipeline class that receives its dependencies. `RAGPipeline` in `src/rag.py` takes its `retriever` and `llm` as constructor arguments, and a `from_env()` class method builds the defaults from environment variables. Results come back as a small `RAGResult` object with a

`to_response()` method, so the response shape is defined in one place, and the same pipeline offers `answer`, `aanswer` and `astream` for sync, async and streaming use. Methods have type hints, for example `_retrieve(self, question: str, top_k: int) -> list[dict]` and `astream(...) -> AsyncIterator[dict]`. This makes the class easy to reuse and easy to test: the tests in `tests/test_rag_pipeline.py` build it with a `FakeRetriever`.

Pattern to refactor: a factory that fails silently. In `src/llm/EmbeddingProviderFactory.py`, `create()` ends with `return None` when the provider name is not recognised, so a typo in the provider name is not reported where it happens. Raising a clear error is a small change:

```
raise ValueError(
    f"Unknown embedding provider: {provider!r}. "
    f"Expected one of: {[e.value for e in EmbeddingEnums]}"
)
```

In the same method, `embedding_size=1024` is hard-coded, `model_name: str = None` could be typed `str | None`, and the `config` passed to `__init__` is stored but not used in that class. Also, the files in `src/llm` (`EmbeddingProviderFactory.py`, `LLMProviderFactory.py`) are named in PascalCase while modules elsewhere in the project use snake_case; renaming them would make the project more consistent.

3. One genuine strength

The thing that impressed me most is that one `RAGPipeline` class is shared across the project instead of every script building its own flow. It is used by the API (`src/routes/root.py`), the BentoML service (`src/serving/bento_service.py`), the monitoring job (`src/monitoring/ragas_monitor.py`), the experiment scripts (`src/mlflow/chunking_ragas_sweep.py` and `src/optimization/compare_llms.py`) and the tests. In production this matters because the answer flow lives in one place, so a fix or a tuning change applies to the API, the service and the monitoring job together instead of drifting apart. The pipeline also carries its own observability hooks (a drift monitor and a tracer are set up in its constructor, and there is a `/drift` endpoint), and the Bento service accepts a `RAG_PIPELINE_FACTORY` setting to plug in a different pipeline for tests or development. These are the details that make a service easier to run and debug after it is deployed.

4. Two improvements

1. Reuse the shared prompt in the evaluation script. `src/evaluation/evaluate_rag.py` has its own copy of the prompt (line 84, inside `build_prompt`), imports `ChatGroq` directly and builds its own `LegalRetriever()` (lines 8 and 197), instead of using `LEGAL_SYSTEM_PROMPT` from `src/rag.py` (line 30). The two prompts already differ: the one in `rag.py` asks for Arabic answers and tells the model to say when an article is repealed, while the evaluation one answers in the language of the question, asks for a concise answer and has no rule about repealed articles. This matters because the file that `evaluate_rag.py` writes (`data/evaluation/rag_results.json`) is the file that `prepare_ragas_dataset.py` reads, taking the `answer` entries from it to build `ragas_dataset.json`. The docstring at the top of `rag.py` says `RAGPipeline` is used by "every evaluation script". The sweep, compare and monitor scripts do use it, but `evaluate_rag.py` does not. The simplest fix is one import (`from src.rag import LEGAL_SYSTEM_PROMPT`), or even better calling `RAGPipeline.from_env().answer(question)`, so the evaluation and the served pipeline cannot drift apart. There may be a reason I am missing, for example that this is an older script.

2. Make the `/health` numbers clearer. `/health` returned `"documents_indexed": 1149`, while the startup log said `indexed 2298 chunks`, and `fetch_data` printed `articles=1149 chunks=2298`.

Someone checking that indexing worked may not know which number to expect. Naming each field for what it counts, or returning both counts, would remove the doubt.

5. Extension idea

I would add a small smoke test that runs in GitHub Actions on every pull request. It would start the stack with `docker compose up`, wait for `/health`, then ask 5 to 10 fixed legal questions through `/ask` and check that each response mentions the article number we expect. This matters because it tells the maintainers right away if a prompt, model or dependency change makes the service stop answering or cite the wrong articles, and it also gives newcomers a working example of a first-run check. To build it, keep the question and expected-article pairs in a small JSON file, write a short Python script that calls `/ask` with a timeout and fails the job when an answer does not mention the expected article, and add a longer nightly run with a bigger question set. The timeout also means a stuck request fails the job instead of hanging forever.

Smaller observations (optional)

- Two classes are named `LegalSource`: one in `src/schemas/sources.py` (line 4) and one in `src/retrieval/schemas.py` (line 6). It may be worth checking whether one can import the other.
- There are two files called `bento_service.py`, one in `src/` and one in `src/serving/`. `bentofile.yaml` and `Dockerfile.bento` both point to the one in `src/serving`. A short note saying which one is current would help newcomers.
- `src/routes/legal.py` re-exports names from `src.rag`, and a comment on line 4 says this is only so that tests and notebooks can keep importing from there. Updating those imports would leave one clear import path.
- Several evaluation scripts have similar names (`evaluate_ragas.py`, `evaluate_ragas_gemini.py`, `evaluate_ragas_gemini_50.py`). `evaluate_ragas.py` already accepts `--dataset`, `--results` and `--provider`, and `docs/COMPLETION_GUIDE.md` says the Gemini scripts are now wrappers and that `evaluate_ragas_gemini_50` is no longer needed. Showing the flags in the README, or removing the old scripts, would make it clearer which one to run.

Thanks again for the project and for the clear structure. Happy to clarify anything above, and good luck with the next steps!